Nearest Centroid, k-Nearest Neighbors, Multilayer Perceptron and Convolutional Neural Networks for Image Classification on the CIFAR-10 Dataset

Sotiris Manolis, 4444

***Abstract*—This report consists of two main parts.**

1) **The first part details the implementation and evaluation of two distance based classification algorithms: the Nearest Centroid Classifier and k-Nearest Neighbors (k-NN) for k=1 and k=3, for the image classification task on the CIFAR-10 dataset. The report covers data loading, preprocessing, dimensionality reduction using Principal Component Analysis (PCA), and the implementation of the algorithms for the classification process. Performance metrics such as prediction accuracy and computation time are also discussed.**
2) **The second part presents different Multilayer Perceptron (MLP) architectures implemented using the PyTorch framework for the same classification task on the CIFAR-10 dataset. Various architectures with different numbers of layers and neurons are explored, as well as different activation functions and optimization techniques. The training process, hyperparameter tuning, and evaluation metrics such as accuracy and loss curves are discussed in detail. Also, the impact of different techniques such as dropout, batch normalization and learning rate scheduling on the performance of the MLP models is analyzed. Finally, the results are compared with those obtained from the distance based algorithms in the first part of the report.**

I. Nearest Centroid and k-Nearest Neighbors Classifiers

Distance based algorithms such as k-Nearest Neighbors (k-NN) and Nearest Centroid are simple yet effective classification methods that rely on the proximity of data points in feature space. This report details the implementation of these algorithms which are eventually trained and tested on the CIFAR-10 dataset, a widely used benchmark in machine learning. However, due to the high dimensionality of the image data (3072 dimensions), dimensionality reduction techniques such as Principal Component Analysis (PCA) are applied to improve computational efficiency and potentially improve classification performance. The object of this report is to analyze the performance of these algorithms in terms of accuracy and computation time and to explore the impact of different methods of data preprocessing and dimensionality reduction on the classification results.

II. Methodology

A. Data Loading

The CIFAR-10 dataset is loaded using TensorFlow's Keras API. The dataset consists of 60.000 32x32x3 color images in total, with 50.000 training images and 10.000 test images, divided into 10 classes, labeled from 0 to 9.

B. Data Preprocessing

First of all, each image is flattened from a (32,32,3) array into a 1D feature vector of D=3072 to facilitate further processing. So, the training set becomes a matrix of shape (50000, 3072) and the test set becomes (10000, 3072). Before applying dimensionality reduction and the testing classification algorithms, the images(training and test sets) are preprocessed. Two methods of preprocessing are implemented: normalization and standardization.

- Normalization: The images are normalized to have pixel values in the range [0.0, 1.0], using a simple Min-Max scaling approach.

$$X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}} \tag{1}$$

  where X is the original pixel value, $X_{min} = 0$ and $X_{max} = 255$ are the minimum and maximum pixel values respectively. Basically, we scale pixel values to the range [0.0, 1.0] by dividing by 255.0. This way, we can ensure that all pixel values contribute equally to the distance computation.
- Standardization: Centers the data to have zero mean and unit variance across the dataset.

$$X_{new} = \frac{X - \mu}{\sigma} \tag{2}$$

  where μ is the mean pixel/feature value across the training set and σ is the standard deviation. It is important to note that the mean and standard deviation are computed only on the training set and then applied to both training and test sets to avoid data leakage.

C. Dimensionality Reduction

To improve computational efficiency, Principal Component Analysis (PCA) using Singular Value Decomposition (SVD) analysis is applied to reduce the dimensionality of the feature space from D(3072 in our case) to a lower dimension d. This happens in two steps:

- We train the PCA model on the training data, finding the top d principal components using SVD
- Apply the trained PCA model to both training and test data to obtain reduced representations

To perform PCA, the following algorithm is used:
PCA finds the directions (principal components) that maximize the variance in the data, allowing us to represent the

Algorithm 1 PCA fit transform

Input: Data matrix $X(N, D)$ where N is the number of samples and D is the feature dimension, and d the number of components we want to keep.
Step 1: Compute the mean of each feature
$X_{meaned} = \frac{1}{N}\sum_{i=1}^{N} X_i$
Step 2: Center the data
$X_{centered} = X - X_{meaned}$
Step 3: Compute SVD
$X_{centered} = U\Sigma V^T$
Step 4: Select top d components from V^T
$V_d = V[:, : d].T$
Step 5: Project data onto the principal components
$X_d = X_{centered} \cdot V_d$
Output: Reduced data matrix X_d, principal components V_d and mean X_{meaned}

data in a lower-dimensional space while retaining most of the information. To obtain these principal components, we perform Singular Value Decomposition (SVD) on the centered data matrix. The SVD provides these directions as the right singular vectors V. By selecting $d < D$ principal components, we achieve dimensionality reduction while preserving most important variations in the data. The same transformation is applied also to the test images to ensure consistency in the feature space (using algorithm 2). It should be noted

Algorithm 2 PCA transform

Input: New data matrix $X(N, D)$, principal components $V_d(d, D)$, mean X_{meaned}
Step 1: Center the new data using the training mean
$X_{centered} = X - X_{meaned}$
Step 2: Project onto the precomputed principal components
$X_d = X_{centered} \cdot V_d$
Output: Reduced data matrix X_d

that d is a hyperparameter that can be tuned based on the trade-off between computational efficiency and classification performance(accuracy).

III. Classification

As previously mentioned, two distance based classification algorithms are implemented: Nearest Centroid Classifier and k-NN for k=1 and k=3. For both algorithms, squared Euclidean distance is used for all distance computations to avoid the computational cost of square root operations, as the relative distances are sufficient for classification. The squared Euclidean distance between two points x and y is calculated as:

$$||a - b||^2 = ||a||^2 + ||b||^2 - 2a^T \cdot b \tag{3}$$

where $||a||^2$ and $||b||^2$ are the squared norms of vectors a and b respectively, and $a^T \cdot b$ is their dot product.

A. Nearest Centroid Classifier

The Nearest Centroid Classifier is one of the simplest classification algorithms. It works by computing the centroid (mean vector) of each class in the training data. During classification, a new sample is assigned to the class whose centroid is closest (in terms of squared Euclidean distance) to the sample. The algorithm is implemented using 2 main steps:

- Compute the centroids for each class (from 0 to 9 as they are labeled in CIFAR-10) from the training data
- For each test sample, compute the squared Euclidean distance to each centroid and assign the class of the nearest centroid

Algorithm 3 Nearest Centroid

Input: Training data X_{train}, training labels y_{train}, test data X_{test}
Step 1: Compute centroids for each class
for each class c **do**
Compute centroid C_c as the mean of all training samples in class c
$C_c = \frac{1}{N_c}\sum_{i=1}^{N_c} X_{train}[i]$
end for
Step 2: For each test sample:
for each test sample in the test set **do**
Compute squared Euclidean distance to each centroid
$distances = ||x_{test} - C_c||^2$
Assign the label of the nearest centroid
$\hat{y} = \arg\min_c distances$
end for
Output: Predicted labels for test data

B. k-Nearest Neighbors (k-NN)

The k-NN algorithm classifies a sample based on the classes of its k nearest neighbors in the training data. In this report, we examine the cases of k=1 and k=3, although the algorithm is implemented in a way that allows for any (positive) value of k not exceeding the number of training samples. The dataset is processed in batches to handle memory constraints effectively. The algorithm consists of the steps described in algorithm 4.

This algorithm efficiently computes distances of the test samples to all training samples in batches, retrieves the k nearest neighbors, and performs majority voting to assign the predicted labels. If multiple labels tie for the most votes, the label of the closest neighbor among the tied labels (candidates) is chosen. This batch processing approach helps manage memory usage effectively while still allowing for accurate k-NN classification. Additionally, as it will be shown in the results section, this batched approach significantly reduces computation time compared to processing all test samples at once without reducing accuracy.

IV. Experimental Results

The performance of the classification algorithms is evaluated using accuracy (number of correct predictions / total predictions) as the primary metric. Experiments are conducted with different configurations, such as:

Algorithm 4 k-NN Prediction

Input: Training data X_{train}, training labels y_{train}, test data X_{test}, number of neighbors k, batch size B
Step 1: For each batch of test samples:
for each batch $X_{test}[i:j]$ **do**
  Compute distances to all training samples
    $dists = ||X_{test}[i:j]||^2 + ||X_{train}||^2 - 2X_{test}[i:j] \cdot X_{train}^T$
  if $k == 1$ **then**
    Assign the label of the nearest neighbor, i.e., the training sample with the minimum distance
  else
    Find the indices of the k nearest neighbors for each test sample, retrieve their labels and their distances
    Perform majority voting among the k neighbors to determine the predicted label
    In case of a tie, choose the label of the closest neighbor among the tied labels
  end if
end for
Output: Predicted labels for test data

- Using flattening and normalization/standardization for data preprocessing
- Varying the number of PCA components d (e.g., 25, 100, 1536) or not applying PCA at all
- Different batch sizes for k-NN prediction (e.g., 500, 300, 5000)

The results are summarized in Table I, showing the accuracy and computation time for each configuration.

The first experiment that was conducted was to run the classification algorithms on the raw flattened data without any preprocessing or PCA on the whole test set(10k samples) in order to establish a baseline for comparison. As expected, the results were poor with an accuracy of only 10% for both Nearest Centroid and k-NN classifiers, as the high dimensionality and lack of preprocessing negatively impacted performance, resulting in long computation times as well($\simeq$ 16 minutes for k-NN).

Nextly, normalization and standardization were applied to the flattened data without PCA, leading to significant improvements in accuracy. The computation times also decreased substantially, especially for k-NN, due to the more compact representation of the data. It is evident that both preprocessing methods improved performance, with standardization slightly outperforming normalization in terms of accuracy, while normalization was marginally faster.

Subsequently, PCA was applied to reduce the dimensionality of the data to 1536 components while retaining the whole test set as batch size, followed by normalization and standardization. This had little to no effect on accuracy for k-NN and Nearest Centroid respectively, compared to the previous experiments without PCA, but computation times for both classifiers decreased further. This indicates that PCA effectively reduced the feature space while retaining important information for classification. When taking into account the computation time of PCA itself, the overall time for Nearest Centroid increased slightly, but for k-NN, the time savings during prediction did not outweigh the PCA computation time, resulting in an increase in total run time.

After applying PCA to halve the dimensionality from 3072 to 1536, the same experiment was repeated with the half of the test set(5000 samples) as batch size. This further reduced computation times for k-NN significantly, while accuracy remained consistent with previous results. At this point it became clear that reducing the batch size for k-NN prediction had a substantial impact on computation time without sacrificing accuracy.

Continuing this trend, experiments were conducted with progressively smaller batch sizes (2500, 1000, 300, 100, 50) while also progressively applying PCA to reduce the dimensionality further (768, 300, 100, 25, 10 components). The results showed that as batch size decreased, the computation time for k-NN prediction decreased significantly, while accuracy remained relatively stable. This demonstrates the effectiveness of batch processing in managing memory usage and speeding up computations without compromising classification performance. Regarding PCA, reducing the number of components led to slight decrease in accuracy for the Nearest Centroid classifier, while k-NN accuracy increased until very low dimensions (10 components) where a more noticeable drop occurred. Also, it was observed that for the Nearest Centroid classifier, the computation time for prediction decreased as the number of PCA components decreased, something that can also be seen for k-NN but to a lesser extent (experiments 16-21). This is expected as lower dimensional data requires less computation for distance calculations.

After reaching a batch size of 50 samples for k-NN prediction and 10 PCA components, it is clear that further reducing batch size and/or PCA components would likely lead to diminishing returns in terms of computation time savings and accuracy loss for both classifiers. Therefore, the values of 100 samples for batch size and 25 components for PCA were chosen as the optimal balance of speed and accuracy. As it can be seen in experiments 20 and 21 the pair achieves the highest accuracy for k-NN classifier for both normalization(38.45% for k=1 and 37.89% for k=3) and standardization(38.26% for k=1 and 37.94% for k=3) while also achieving the lowest computation times(3.65sec and 6.75sec for normalization, 3.73sec and 6.19sec for standardization).

Finally, to validate the effectiveness of PCA for k-NN prediction, experiments 26-27 were conducted using the entire test set (10k samples) as the batch size achieving the highest accuracy for k-NN classifier(38.45% for k=1 and 37.89% for k=3 with normalization,38.26% for k=1 and 37.94% for k=3 with standardization) but with a significantly reduced computation time(71.14sec and 91.38sec for normalization,78.09sec and 100.28sec for standardization) compared to the baseline experiments without PCA. So, it is safe to conclude that PCA is highly effective in reducing computation time for k-NN prediction while maintaining and improving accuracy, especially when combined with batch processing. Regarding the Nearest Centroid classifier, while PCA did reduce computation time during prediction, the

TABLE I
CLASSIFICATION ACCURACY ON CIFAR-10 DATASET

	Preprocessing	PCA (dimensions, time)	Nearest Centroid (compute, predict, accuracy)	Batch Size	1-NN (accuracy, time)	3-NN (accuracy, time)
1	Flattening	-	0.29sec, 0.32sec, 10%	whole set(10k)	945.12sec, 10%	946.35sec, 10%
2	Flattening,Normalization	-	0.32sec, 0.11sec, 27.74%	whole set(10k)	115.43sec, 35.39%	127.72sec, 33.03%
3	Flattening,Standardization	-	0.35sec, 0.10sec, 28.12%	whole set(10k)	110.83sec, 35.67%	149.66sec, 33.08%
4	Flattening,Normalization	1536, 115.4sec	0.2sec, 0.056sec, 27.74%	whole set(10k)	90.32sec, 35.41%	116.66sec, 33.08%
5	Flattening,Standardization	1536, 108.08sec	0.19sec, 0.058sec, 28.12%	whole set(10k)	77.55sec, 35.71%	98.8sec, 33.11%
6	Flattening,Normalization	1536, 104.4sec	0.21sec, 0.07sec, 27.74%	5000	37.38sec, 35.41%	41.89sec, 33.08%
7	Flattening,Standardization	1536, 102.2sec	0.23sec, 0.06sec, 28.12%	5000	46.12sec, 35.71%	70.84sec, 33.11%
8	Flattening,Normalization	1536, 102.5sec	0.22sec, 0.06sec, 27.74%	2500	13.73sec, 35.41%	20.52sec, 33.08%
9	Flattening,Standardization	1536, 100.8sec	0.23sec, 0.06sec, 28.12%	2500	12.77sec, 35.71%	20.62sec, 33.11%
10	Flattening,Normalization	768, 101.5sec	0.11sec, 0.03sec, 27.75%	2500	8.28sec, 35.70%	18.16sec, 33.20%
11	Flattening,Standardization	768, 99.6sec	0.13sec, 0.03sec, 28.12%	2500	8.36sec, 35.87%	15.91sec, 33.29%
12	Flattening,Normalization	768, 100.2sec	0.12sec, 0.03sec, 27.75%	1000	8.41sec, 35.70%	11.01sec, 33.20%
13	Flattening,Standardization	768, 112.9sec	0.14sec, 0.03sec, 28.12%	1000	8.77sec, 35.87%	11.08sec, 33.29%
14	Flattening,Normalization	300, 98.4sec	0.05sec, 0.02sec, 27.75%	300	5.18sec, 36.81%	7.93sec, 34.40%
15	Flattening,Standardization	300, 106.2sec	0.05sec, 0.02sec, 28.09%	300	5.30sec, 36.96%	8.22sec, 34.84%
16	Flattening,Normalization	300, 96.4sec	0.04sec, 0.01sec, 27.75%	100	6.09sec, 36.81%	8.04sec, 34.40%
17	Flattening,Standardization	300, 98.2sec	0.04sec, 0.01sec, 28.09%	100	5.93sec, 36.96%	8.74sec, 34.84%
18	Flattening,Normalization	100, 100.9sec	0.03sec, 0.009sec, 27.67%	100	4.29sec, 38.50%	7.29sec, 36.68%
19	Flattening,Standardization	100, 102.3sec	0.03sec, 0.009sec, 28.07%	100	3.872sec, 38.64%	6.44sec, 36.88%
20	Flattening,Normalization	25, 107.7sec	0.04sec, 0.007sec, 27.51%	100	3.65sec, 38.45%	6.75sec, 37.89%
21	Flattening,Standardization	25, 102.2sec	0.03sec, 0.005sec, 27.85%	100	3.73sec, 38.26%	6.19sec, 37.94%
22	Flattening,Normalization	25, 110.5sec	0.04sec, 0.007sec, 27.51%	50	5.30sec, 38.45%	8.26sec, 37.89%
23	Flattening,Standardization	25, 104.3sec	0.04sec, 0.006sec, 27.85%	50	5.22sec, 38.26%	9.45sec, 37.94%
24	Flattening,Normalization	10, 115.2sec	0.04sec, 0.005sec, 26.37%	50	4.20sec, 30.71%	8.48sec, 30.87%
25	Flattening,Standardization	10, 108.7sec	0.28sec, 0.006sec, 26.56%	50	3.86sec, 30.86%	7.09sec, 30.95%
26	Flattening,Normalization	25, 96.4sec	0.28sec, 0.006sec, 27.51%	10k	71.14sec, 38.45%	91.38sec, 37.89%
27	Flattening,Standardization	25, 98.2sec	0.29sec, 0.005sec, 27.85%	10k	78.09sec, 38.26%	100.28sec, 37.94%
28	Flattening,Normalization	-	0.54sec, 0.11sec, 27.74%	100	29.61sec, 35.39%	30.14sec, 33.03%
29	Flattening,Standardization	-	0.54sec, 0.13sec, 28.12%	100	28.95sec, 35.67%	30.26sec, 33.08%

overall time increased when considering the PCA computation time itself. More importantly, PCA did not improve accuracy for Nearest Centroid, indicating that while PCA is beneficial for k-NN, it may not be as effective for simpler classifiers like Nearest Centroid.

The final two experiments(28-29) were conducted to validate the effectiveness of batch processing for k-NN prediction without applying PCA. Using a batch size of 100 samples for k-NN prediction on the normalized and standardized flattened data respectively, the computation times were significantly reduced(29.61sec and 30.14sec for normalization, 28.95sec and 30.26sec for standardization) compared to the baseline experiments without PCA(115.43sec and 127.72sec for normalization, 110.83sec and 149.66sec for standardization) while maintaining the same accuracy (35.39% and 33.03% for normalization, 35.67% and 33.08% for standardization). This confirms that batch processing is an effective strategy for managing memory usage and speeding up computations for k-NN classification without sacrificing accuracy.

V. CONCLUSION

This report presented the implementation and evaluation of the Nearest Centroid Classifier and k-Nearest Neighbors (k-NN) algorithms for image classification on the CIFAR-10 dataset. The study explored various data preprocessing techniques, including normalization and standardization, as well as dimensionality reduction using Principal Component Analysis (PCA). The results demonstrated that both preprocessing methods significantly improved classification accuracy compared to using raw flattened data. In particular, standardization slightly outperformed normalization in terms of accuracy, while normalization was marginally faster. Furthermore, PCA effectively reduced computation time for k-NN prediction while maintaining or improving accuracy, especially when combined with batch processing. For the Nearest Centroid classifier, PCA reduced computation time during prediction, but the overall time increased when considering the PCA computation time itself, and it did not improve accuracy, thus indicating that PCA may not be as effective for simpler classifiers. For k-NN though, PCA proved to be highly beneficial. The optimal configuration (batch size=100, PCA =25) achieved an accuracy of 38.45% for 1-NN and 37.89% for 3-NN with a computation time of just a few seconds for the predicition phase and around 100 seconds including PCA computation time which is a significant improvements compared to the baseline experiments without PCA. Compared even to the experiments with batch processing and only normalization/standardizationbut without PCA, the chosen configuration still achieved higher accuracy justifying the computational cost of PCA.
In conclusion, the combination of effective data preprocessing (normalization/standardization) and dimensionality reduction using PCA, and batch processing for k-NN prediction proved to be a powerful approach for image classification on the CIFAR-10 dataset, achieving notable improvements both for accuracy and computational efficiency.

VI. Multilayer Perceptron

A Multilayer Perceptron (MLP) is a fully connected feed-forward neural network (FNN) that consists of an input layer, one or more hidden layers and an output layer. Each layer contains multiple neurons and each one of them is connected to every neuron in the next layer. These fully connected layers transform input data from one dimensional space to another by applying a linear transformation followed by a non-linear activation function.

- **Input Layer:** The input layer receives the input data. In our case, each CIFAR-10 image is flattened into a 1D vector of size 3072 (32x32x3).
- **Hidden Layers:** The hidden layers perform computations on the input data. Each neuron in a hidden layer applies a weighted sum of its inputs followed by a non-linear activation function (e.g. Sigmoid, ReLU). The number of hidden layers and neurons per layer are hyperparameters that can be tuned.
- **Output Layer:** The output layer produces the final predictions. For CIFAR-10 classification, the output layer has 10 neurons (one for each class) and uses the Softmax activation function to output probabilities for each class.

The MLP neural network is trained using backpropagation, a supervised learning technique, combined with an optimization algorithm like Stochastic Gradient Descent (SGD) or Adam. The goal of training is to minimize a loss function, such as Cross-Entropy Loss for our classification task, by adjusting the weights and biases of the network. Training consists of two main phases:

- **Forward Pass:** The input data is passed through the network layer by layer. Each beuron computes a weighted sum of its inputs

$$z = \sum_i w_i x_i + b \tag{4}$$

where w_i are the weights, x_i are the inputs and b is the bias. Then, the weighted sum is passed through a non-linear activation function, such as:

  - Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{5}$$

  - ReLU (Rectified Linear Unit):

$$ReLU(z) = \max(0, z) \tag{6}$$

This process continues through all hidden layers until reaching the output layer, where the Softmax function is applied to obtain class probabilities:

$$Softmax(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}} \tag{7}$$

where z_j is the output of the jth neuron in the output layer. **Note:** In our experiments, the Softmax activation function is not applied at the output layer, as the Cross-Entropy Loss function in PyTorch framework combines the Softmax operation internally for numerical stability.

- **Loss Computation and Optimization:** After the forward pass, the loss is computed using the predicted outputs and the true labels. The Cross-Entropy Loss for multi-class classification is given by:

$$L = -\sum_i y_i \log(\hat{y}_i) \tag{8}$$

where y_i is the true label (one-hot encoded) and $\hat{y}_i$ is the predicted probability for class i.

- **Backward Pass:** The loss is computed based on the predictions and true labels. The gradients of the loss with respect to the weights and biases are calculated using the chain rule of calculus (backpropagation). The error then is propagated backward through the network, layer by layer and the gradients are computed for each weight and bias.
- **Weight Update:** The weights and biases are updated using the computed gradients and an optimization algorithm like SGD or Adam. For example, in SGD, the weights are updated as follows:

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i} \tag{9}$$

where η is the learning rate and $\frac{\partial L}{\partial w_i}$ is the gradient of the loss with respect to weight w_i.

Moreover, various techniques are used to improve the performance and generalization of the MLP model such as:

- **Dropout:** A regularization technique where randomly selected neurons are ignored during training. This prevents overfitting by reducing co-adaptation of neurons.
- **Batch Normalization:** A technique that normalizes the inputs of each layer to have zero mean and unit variance. This helps stabilize and accelerate training.
- **Learning Rate Scheduling:** Adjusting the learning rate during training, often reducing it as training progresses to allow for finer convergence.
- **Data Augmentation:** Generating additional training data by applying random transformations (e.g., rotations, flips) to the original images, which helps improve generalization.
- **Weight Decay:** A regularization technique that adds a penalty to the loss function based on the magnitude of the weights, encouraging smaller weights and reducing overfitting.

VII. Experiment Pipeline

In this section, the experimental setup for training and evaluating the MLP models on the CIFAR-10 dataset is described. At first, the train dataset is loaded and we calculate the mean and standard deviation of the pixel values across all images to use for normalization. Then, the train and test datasets are loaded with appropriate transformations applied. Nextly, the train dataset is split into training and validation sets using a validation ratio of 0.9. This means that 90% of the training data is used for training the model, while the remaining 10% is used for validation during training to monitor performance and prevent overfitting, meaning the training set contains 45000 examples and the validation set contains 5000 examples and both sets are used to train and evaluate the MLP models. Data loaders are created for the training, validation, and test sets

with a batch size of 64. The training data loader shuffles the data to ensure that the model sees different examples in each epoch, while the validation and test data loaders do not shuffle the data.

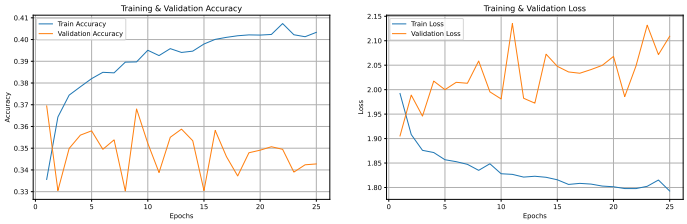
The MLP models are defined with varying architectures, including different numbers of hidden layers and neurons, as well as different activation functions (Sigmoid and ReLU). The models are trained using the Adam optimizer with a learning rate of 0.001 (or 0.0001 in some experiments) and Cross-Entropy Loss as the loss function. Training is performed for 25 epochs, with the model's performance evaluated on the validation set after each epoch. The training and validation accuracy and loss are recorded for analysis. After training, the best model (based on validation accuracy) is evaluated on the test set to obtain the final test accuracy and loss.

A. Baseline MLP Model

The first experiment involves training a baseline MLP model with the following architecture:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Output Layer: 10 neurons (one for each class)

No hidden layers are used in this baseline model. The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss as the loss function. As expected, the baseline model achieves a low accuracy of 35.69% on the test set, indicating that a more complex architecture is needed to capture the underlying patterns in the data.



(a) (b)

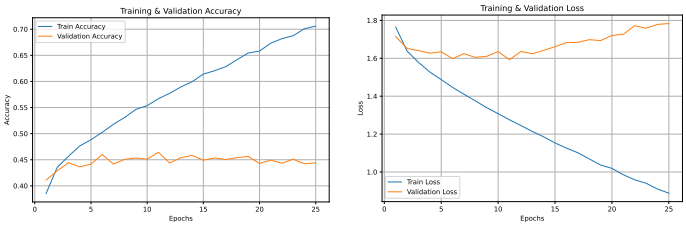
Fig. 1. Baseline MLP training metrics across epochs.

B. MLP with 1 Hidden Layer

Building upon the baseline model, a hidden layer with 512 neurons and Sigmoid/ReLU activation function is added. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with Sigmoid/ReLU activation
- Output Layer: 10 neurons with Softmax activation

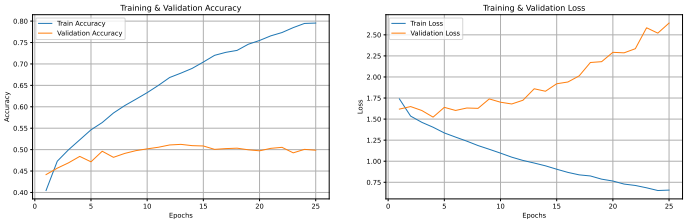
The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss as the loss function. Firstly, the Sigmoid activation function is used in the hidden layer. The model achieves an accuracy of 46.3% and a loss of 1.590 on the test set and the training and validation loss decrease steadily over for the first 5 epochs before plateauing, indicating that the model is learning but may be struggling to generalize further. After that, the training loss continues to decrease slightly while the validation loss starts to increase, suggesting overfitting.



(a) (b)

Fig. 2. MLP with 1 hidden layer (Sigmoid) training metrics across epochs.

Nextly, the ReLU activation function is used in the hidden layer. The model achieves a higher accuracy of 47.89% and a loss of 1.514 on the test set. However, the validation loss starts to increase after 5 epochs, indicating potential overfitting despite the training loss decreasing steadily. Moreover, the validation accuracy plateaus around 50%, suggesting that the model may not be generalizing well to unseen data. Out of



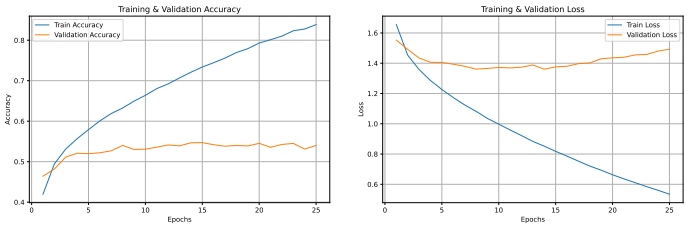
(a) (b)

Fig. 3. MLP with 1 hidden layer (ReLU) training metrics across epochs.

these two experiments, the ReLU activation function performs better. Therefore, further experiments are conducted using the ReLU activation function in the hidden layer. Now the learning rate is reduced to 0.0001 to allow for finer weight updates and potentially better convergence. After retraining the ReLU model for 25 epochs with a learning rate of 0.0001, results show a significant improvement in accuracy, 53.86% on the test set and a loss of 1.358. Train accuracy in this case reaches up to 83%, while validation accuracy plateaus around 54%, indicating better generalization. Also, validation loss starts increasing later in the training and shows a more stable behiaviour for this learning rate compared to the previous experiment with a higher learning rate of 0.001, suggesting that the model is less prone to overfitting. This indicates that lowering the learning rate has a positive impact on the model's performance. Before moving on to more complex architectures, we are going to conduct experiments with different techniques to improve the performance and generalization of the MLP model such as Dropout, Batch Normalization and Learning Rate Scheduling.

C. MLP with Dropout

To mitigate overfitting observed in previous experiments, Dropout is introduced after the hidden layer with a dropout rate



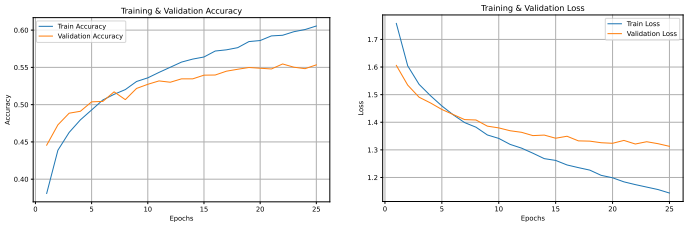
(a) (b)

Fig. 4. MLP with 1 hidden layer (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

of 0.5. This means that during training, 50% of the neurons in the hidden layer are randomly deactivated in each forward pass. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with ReLU activation
- Dropout Layer: Dropout rate of 0.5
- Output Layer: 10 neurons

The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.0001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 55.12% and a loss of 1.294 on the test set. The training accuracy reaches only up to 60.55%, while validation accuracy peaks at around 55.5%. Both training and validation loss decrease steadily throughout the epochs without significant divergence, indicating that Dropout effectively reduces overfitting and improves generalization. This suggests that incorporating Dropout helps the model learn more robust features by preventing reliance on specific neurons. In conclusion, the addition of Dropout has a positive impact on the model's performance and perhaps suggest that training for more epochs could yield even better results, since for 25 epochs there are no signs of overfitting.



(a) (b)

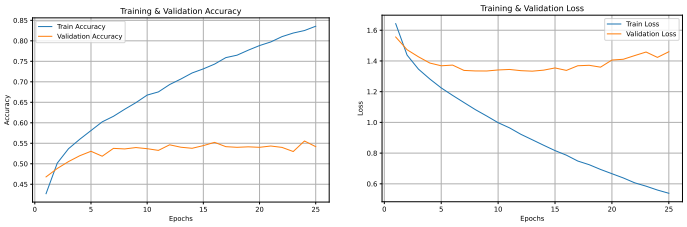
Fig. 5. MLP with Dropout (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

D. MLP with Batch Normalization

To further improve training stability and convergence speed, Batch Normalization is added after the hidden layer. This technique normalizes the inputs to the hidden layer, ensuring that they have zero mean and unit variance. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with ReLU activation
- Batch Normalization Layer
- Output Layer: 10 neurons

The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.0001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 54.06% and a loss of 1.322 on the test set. The training accuracy reaches up to 83.56%, while validation accuracy plateaus around 55%. Training loss decreases steadily, while validation loss starts to increase slightly after 15 epochs, indicating mild overfitting.



(a) (b)

Fig. 6. MLP with Batch Normalization (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

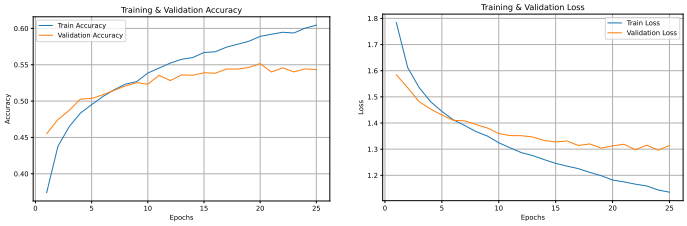
This suggests that while Batch Normalization helps stabilize training and improve convergence, it may not be sufficient alone to fully prevent overfitting in this case. Overall, the addition of Batch Normalization has a positive impact on the model's performance, but further regularization techniques may be needed to enhance generalization. Dropout seems to be more effective than Batch Normalization for reducing overfitting in this particular setup, leading to the next experiment, the combination of both techniques.

E. MLP with Dropout and Batch Normalization

To leverage the benefits of both Dropout and Batch Normalization, a model is created that incorporates both techniques. Dropout is applied after the hidden layer with a dropout rate of 0.5, and Batch Normalization is applied before the activation function. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.5
- Output Layer: 10 neurons

The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.0001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 54.72% and a loss of 1.275 on the test set. The training accuracy reaches up to 60.45%, while validation accuracy peaks at around 55.16%. Both training and validation loss decrease steadily throughout the epochs without significant divergence, indicating that the combination of Dropout and Batch Normalization effectively reduces overfitting and improves generalization. This suggests that incorporating both techniques helps the model learn more robust features by preventing reliance on specific neurons.



(a) (b)

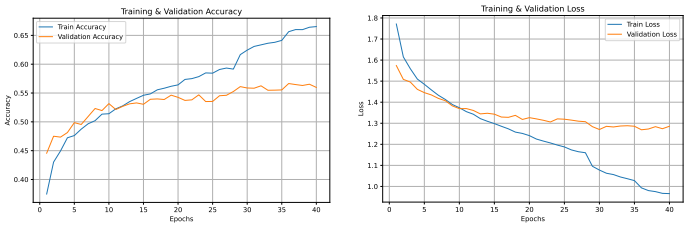
Fig. 7. MLP with Dropout and Batch Normalization (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

F. Learning Rate Scheduling

To further enhance the training process, Learning Rate Scheduling is implemented. The learning rate starts at 0.001 and is reduced by a factor of 0.5 every time the validation loss plateaus for 4 epochs. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.5
- Output Layer: 10 neurons

The model is trained for 40 epochs using the Adam optimizer and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 56.53% and a loss of 1.259 on the test set, the best performance observed. The training accuracy reaches up to 66%, while validation accuracy peaks at around 56.65%. Both training and validation loss decrease steadily throughout the epochs, indicating that Learning Rate Scheduling effectively enhances training stability and convergence. This suggests that dynamically adjusting the learning rate based on validation performance helps the model learn more effectively. By reducing the learning rate when the validation loss plateaus, the model can make finer adjustments to the weights, leading to improved generalization. By looking at the 8b, it is observed that the validation loss does



(a) (b)

Fig. 8. MLP with Learning Rate Scheduling (ReLU, batch size=64) training metrics across epochs.

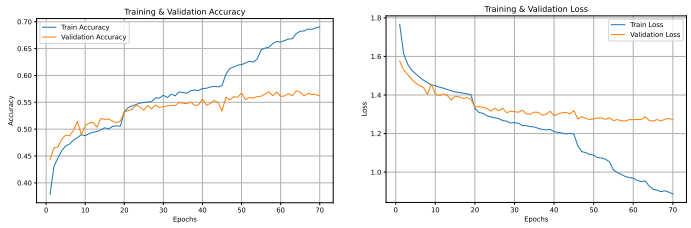
not increase during training, indicating that the model is not overfitting and suggesting the need for more training epochs. This suggests that Learning Rate Scheduling, in combination with Dropout and Batch Normalization, is effective in improving the model's performance and generalization. Overall, the addition of Learning Rate Scheduling has a positive impact on the model's performance, leading to the best results observed so far.

G. Weight Decay

To further regularize the model and prevent potential overfitting, Weight Decay (L2 regularization) is introduced during training. A weight decay factor of 0.0001 is applied. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.5
- Output Layer: 10 neurons

The model is trained for 70 epochs this time using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 56.36% and a loss of 1.257 on the test set. The training accuracy reaches up to 69%, while validation accuracy peaks at around 57%. Once again, both training and validation loss decrease steadily throughout the epochs without significant divergence, but validation accuracy plateaus around 57%, indicating that while Weight Decay helps regularize the model, the model may still be struggling to generalize further due to the limited capacity of a single hidden layer.



(a) (b)

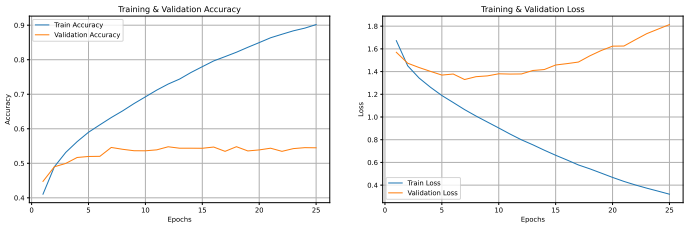
Fig. 9. MLP with Weight Decay (ReLU, lr=0.001, batch size=64) training metrics across epochs.

H. MLP with 2 Hidden Layers

To further enhance the model's capacity to learn complex patterns, a second hidden layer with 250 neurons and ReLU activation function is added. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 512 neurons with ReLU activation
- Hidden Layer 2: 250 neurons with ReLU activation
- Output Layer: 10 neurons

The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.0001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 54.09% and a loss of 1.325 on the test set. The training accuracy reaches up to 90%, while validation accuracy plateaus around 54.5%. Despite the slight increase in accuracy compared to the basic 1 hidden layer model, the validation loss starts increasing after 10 epochs, indicating overfitting. This suggests that while adding a second hidden



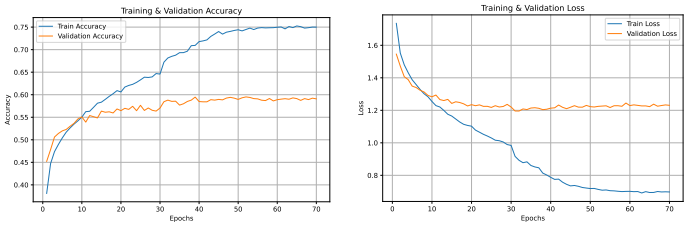
(a) (b)

Fig. 10. MLP with 2 hidden layers (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

layer increases the model's capacity, it also makes it more prone to overfitting. To address the overfitting issue, we apply the same techniques used previously, such as Dropout, Batch Normalization, Weight Decay and Learning Rate Scheduling. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 2: 250 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Output Layer: 10 neurons

The model is trained for 70 epochs using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 58.2% and a loss of 1.196 on the test set, the best performance observed across all experiments. The training accuracy reaches up to 75%, while validation accuracy peaks at around 59.5%. Training loss decreases steadily throughout the epochs, while validation loss starts to increase slightly after 30 epochs, indicating mild overfitting, although the model generalizes better than previous architectures. Also, the validation accuracy shows a steady increase up to around 59.5%, suggesting that the model is learning effectively from the data, but it plateaus after that point, indicating that further improvements may require more complex architectures.



(a) (b)

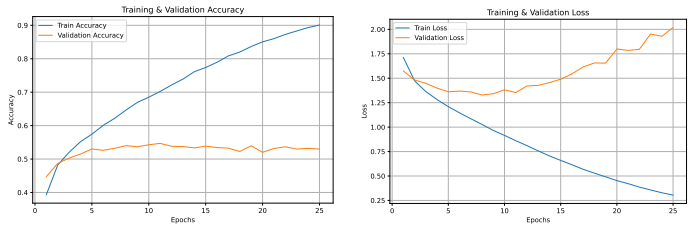
Fig. 11. MLP with 2 hidden layers and advanced techniques (ReLU, lr=0.001, batch size=64) training metrics across epochs.

I. MLP with 3 Hidden Layers

To further increase the model's capacity, a third hidden layer with 100 neurons and ReLU activation function is added. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 512 neurons with ReLU activation
- Hidden Layer 2: 250 neurons with ReLU activation
- Hidden Layer 3: 100 neurons with ReLU activation
- Output Layer: 10 neurons

The model is trained for 25 epochs using the Adam optimizer with a learning rate of 0.0001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 54.7% and a loss of 1.321 on the test set, an improvement over the basic 2 hidden layer model but still not surpassing the best performance observed. The training accuracy reaches up to 90%, while validation accuracy plateaus around 54%. The validation loss starts increasing after 10 epochs, clearly indicating overfitting. Now, we apply the same techniques used



(a) (b)

Fig. 12. MLP with 3 hidden layers (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

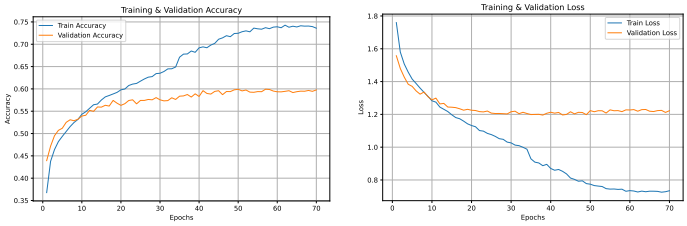
previously, such as Dropout, Batch Normalization, Weight Decay and Learning Rate Scheduling (we use the ReduceLROnPlateau scheduler with patience=5 and factor=0.5). The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 2: 250 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 3: 100 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.2
- Output Layer: 10 neurons

The model is trained for 70 epochs using the Adam optimizer with a learning rate of 0.001 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 58.5% and a loss of 1.190 on the test set. The training accuracy reaches up to 73.6%, while validation accuracy peaks at almost 60%. Even thoguh this model achieves a slightly higher accuracy than the enhanced 2 hidden layer model, the validation accuracy plateaus around 60%, and the validation loss plateaus around 1.2. This suggests that while adding a third hidden layer increases the model's capacity, it does not significantly improve generalization meaning that the model overfits the training data.

J. MLP with 4 Hidden Layers

To further explore the impact of model depth, a hidden layer with 1024 neurons and ReLU activation function is added. The



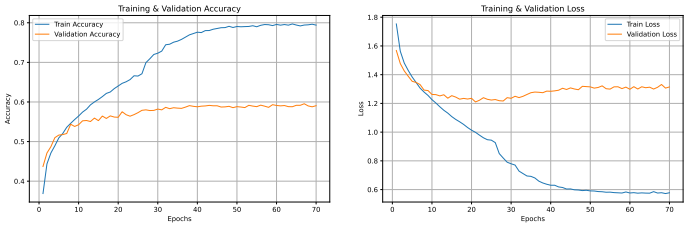
(a) (b)

Fig. 13. MLP with 3 hidden layers (ReLU, lr=0.0001, batch size=64) training metrics across epochs.

architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 1024 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 2: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 3: 250 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.2
- Hidden Layer 4: 100 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.1
- Output Layer: 10 neurons

The model is trained for 70 epochs using the Adam optimizer with a learning rate of 0.001, ReduceLROnPlateau scheduler with patience=5 and factor=0.5 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 57.46% and a loss of 1.192 on the test set. The training accuracy reaches up to almost 80%, while validation accuracy peaks at around 59%. Similar to the previous experiments with



(a) (b)

Fig. 14. MLP with 4 hidden layers (ReLU, lr=0.001, batch size=64) training metrics across epochs.

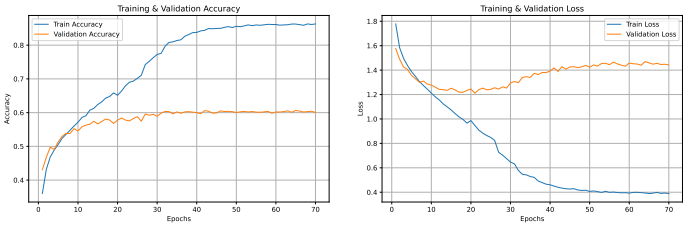
deeper architectures, the validation accuracy plateaus around 59%, and the validation loss plateaus around 1.2 and after around 30 epochs it starts to increase, indicating that the model is overfitting the training data.

K. MLP with 5 Hidden Layers

Finally, a model with 5 hidden layers is created to further investigate the impact of depth on performance. The architecture of this model is as follows:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 2048 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 2: 1024 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 3: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 4: 250 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.2
- Hidden Layer 5: 100 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.1
- Output Layer: 10 neurons

The model is trained for 70 epochs using the Adam optimizer with a learning rate of 0.001, ReduceLROnPlateau scheduler with patience=5 and factor=0.5 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 57.89% and a loss of 1.213 on the test set. The training accuracy reaches up to 86%, while validation accuracy peaks at around 60.5%. Once again, the validation accuracy plateaus



(a) (b)

Fig. 15. MLP with 5 hidden layers (ReLU, lr=0.001, batch size=64) training metrics across epochs.

around 60.5%, and the validation loss plateaus around 1.2 and after around 25 epochs it starts to increase, indicating that the model is overfitting the training data.

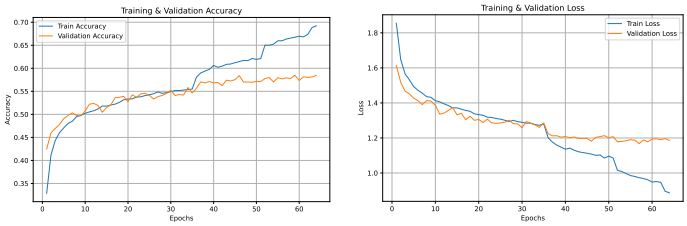
At this stage, we introduce an early stopping mechanism to prevent overfitting. The training process is halted if the validation loss does not improve for an certain number of consecutive epochs.

To further experiment with the model's architecture, we can try different configurations such as varying the number of neurons in each hidden layer. In this spirit, we design a model with 5 hidden layers with the following architecture:

- Input Layer: 3072 neurons (flattened CIFAR-10 images)
- Hidden Layer 1: 1024 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 2: 512 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 3: 256 neurons with ReLU activation
- Batch Normalization Layer

- Dropout Layer: Dropout rate of 0.4
- Hidden Layer 4: 128 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.3
- Hidden Layer 5: 64 neurons with ReLU activation
- Batch Normalization Layer
- Dropout Layer: Dropout rate of 0.2
- Output Layer: 10 neurons

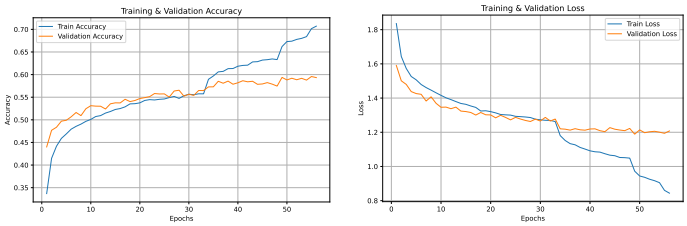
The model is trained for 120 epochs, but stops after 65 due to early stopping (with 8 epochs patience), using the Adam optimizer with a learning rate of 0.001, ReduceLROnPlateau scheduler with patience=5 and factor=0.5 and Cross-Entropy Loss as the loss function. This time, the model achieves an accuracy of 58.15% and a loss of 1.174 on the test set. The training accuracy reaches up to 78%, while validation accuracy peaks at around 59.8%.



(a) (b)

Fig. 16. MLP with 5 hidden layers and alternate architecture (ReLU, lr=0.001, batch size=64) training metrics across epochs.

Now, we are going to experiment with the same architecture but we will remove the ReLU activation function from the last hidden layer. The architecture of this model as the one before. After training for 57 epochs (using early stopping with 8 epochs patience, originally it was set to train for 300), the model achieves an accuracy of 58.26% and a loss of 1.187 on the test set. This indicates that removing the activation function from the last hidden layer has a marginal positive impact on the model's performance. As we can see from the

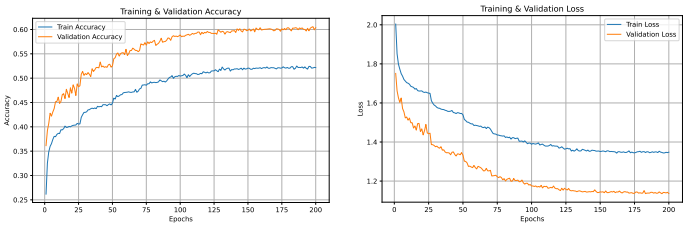


(a) (b)

Fig. 17. MLP with 5 hidden layers and alternate architecture without ReLU in last hidden layer (lr=0.001, batch size=64) training metrics across epochs.

above figures, the validation loss does not increase for the 57 epochs it trained, indicating that the model is not overfitting and suggesting that training for more epochs could yield even better results.

Because this is the best model we have obtained so far, we are going to further experiment with this architecture by training it on augmented data, hoping to improve its generalization capabilities. Data augmentation techniques such as random cropping, horizontal flipping, and normalization are applied to the training dataset to artificially increase the diversity of the training data. We train the model for 200 epochs using the Adam optimizer with a learning rate of 0.001 and a weight decay of 0.0001, ReduceLROnPlateau scheduler with patience=5 and factor=0.5 and Cross-Entropy Loss as the loss function. After training, the model achieves an accuracy of 58.65% and a loss of 1.156 on the test set, slightly better than before and the best performance observed across all MLP experiments. The training accuracy reaches up to around



(a) (b)

Fig. 18. MLP with 5 hidden layers and alternate architecture without ReLU in last hidden layer trained on augmented data (lr=0.001, batch size=64) training metrics across epochs.

52%, while validation accuracy peaks at around 60.5%. This suggests that data augmentation has a positive impact on the model's performance and generalization, helping it learn more robust features from the data. Regarding the loss curves, both training and validation loss decrease steadily throughout the epochs without significant divergence, indicating that the model is not overfitting and is learning effectively from the augmented data.

VIII. Convolutional Neural Networks (CNNs)

To further improve performance on the CIFAR-10 dataset, we explore Convolutional Neural Networks (CNNs), which are specifically designed to handle image data. CNNs utilize convolutional layers to automatically learn spatial hierarchies of features from input images, making them more effective for image classification tasks compared to MLPs. Basically, CNNs work by applying a series of convolutional filters to the input images, which helps in capturing local patterns such as edges, textures, and shapes. These learned features are then passed through pooling layers to reduce dimensionality and retain important information. Finally, fully connected layers are used to make predictions based on the extracted features.

A. First CNN Architecture

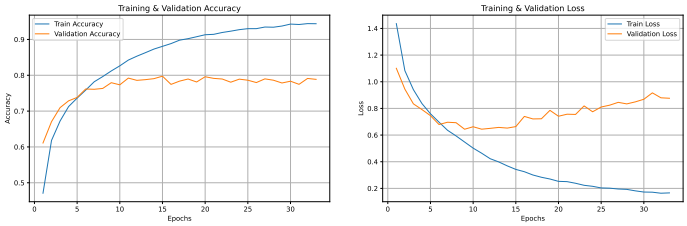
Below is the implementation of the first CNN architecture used for CIFAR-10 classification:

The first CNN architecture designed consists of the following layers:

- Convolutional Layer 1: 32 filters, kernel size 3x3, ReLU activation, Batch Normalization, Max Pooling (2x2)
- Convolutional Layer 2: 64 filters, kernel size 3x3, ReLU activation, Batch Normalization, Max Pooling (2x2)

- Convolutional Layer 3: 128 filters, kernel size 3x3, ReLU activation, Batch Normalization, Max Pooling (2x2)
- Fully Connected Layer 1: 512 neurons, ReLU activation, Dropout (0.5)
- Fully Connected Layer 2: 256 neurons, ReLU activation, Dropout (0.3)
- Output Layer: 10 neurons (one for each CIFAR-10 class)

The model is trained for 50 epochs on the original CIFAR-10 dataset using the Adam optimizer with a learning rate of 0.001, weight decay of 0.0001, Cross-Entropy Loss as the loss function and early stopping with patience of 25 epochs. As for the learning rate scheduler, we use Cosine Annealing. Because of the early stopping mechanism, the training halts after only 34 epochs. Despite that, the model achieves an accuracy of 78.49% and a loss of 0.636 on the test set. The training

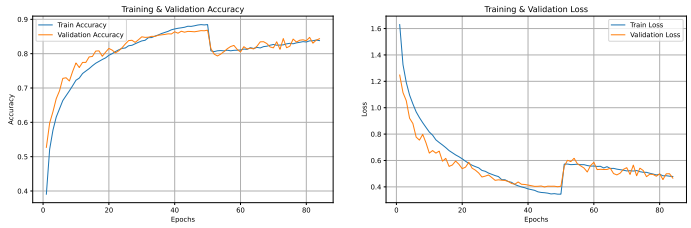


(a) (b)

Fig. 19. First CNN architecture (lr=0.001, batch size=64) training metrics across epochs.

accuracy reaches up to 89%, while validation accuracy peaks at around 79.5%. Training loss decreases steadily throughout the epochs, while validation loss starts to increase after 15 epochs, indicating overfitting. This suggests that while the CNN architecture is more effective in learning from the data compared to the best-performing MLP, further regularization techniques or data augmentation may be needed to enhance generalization.

For the next experiment, we will train the same CNN architecture on augmented data(using random cropping, horizontal flipping, and normalization), hoping to improve its generalization capabilities. Specifically, we train the model for 300 epochs using the Adam optimizer with a learning rate of 0.001, weight decay of 5e-4 (0.0005), Cross-Entropy Loss as the loss function and early stopping with patience of 40 epochs. As for the learning rate scheduler, we use Cosine Annealing but with warm restarts this time. After training for 84 epochs (early stopping), the model achieves an accuracy of 86.15% and a loss of 0.426 on the test set, significantly better than before. The training accuracy reaches up to around 87%, while validation accuracy peaks at around 86-87%. Based on the loss curves, both training and validation loss decrease steadily throughout the epochs, until the scheduler restarts the learning rate, causing a temporary increase in loss, which if not for early stopping, might have led to further overall decrease. However, overall, the model shows improved generalization due to training on augmented data and is not overfitting, meaning that it is learning effectively from the augmented data.



(a) (b)

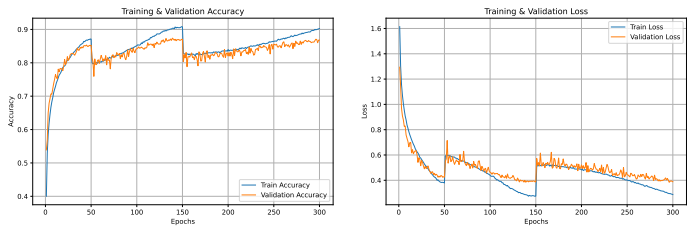
Fig. 20. First CNN architecture trained on augmented data (lr=0.001, batch size=64) training metrics across epochs.

B. *Second CNN Architecture*

For our final experiment, building on the previous architecture, we introduce dropout in the convolutional layers as well, hoping to further improve generalization. The architecture of this model is as follows:

- Convolutional Layer 1: 32 filters, kernel size 3x3, ReLU activation, Batch Normalization, Dropout (0.1), Max Pooling (2x2)
- Convolutional Layer 2: 64 filters, kernel size 3x3, ReLU activation, Batch Normalization, Dropout (0.1), Max Pooling (2x2)
- Convolutional Layer 3: 128 filters, kernel size 3x3, ReLU activation, Batch Normalization, Dropout (0.1), Max Pooling (2x2)
- Fully Connected Layer 1: 512 neurons, ReLU activation, Dropout (0.4)
- Fully Connected Layer 2: 256 neurons, ReLU activation, Dropout (0.2)
- Output Layer: 10 neurons (one for each CIFAR-10 class)

The model is trained for 300 epochs on the augmented CIFAR-10 dataset using the Adam optimizer with a learning rate of 0.001, weight decay of 5e-4 (0.0005), Cross-Entropy Loss as the loss function and early stopping with patience of 40 epochs. As for the learning rate scheduler, we use Cosine Annealing with warm restarts. After training for 300 epochs, the model achieves an accuracy of 87.4% and a loss of 0.381 on the test set, slightly better than before and the best performance observed across all experiments. The training



(a) (b)

Fig. 21. Second CNN architecture trained on augmented data (lr=0.001, batch size=64) training metrics across epochs.

accuracy reaches up to around 90%, while validation accuracy peaks at around 87-88%. Based on the loss curves, both training and validation loss decrease steadily throughout the

epochs, until the scheduler restarts the learning rate, causing a temporary increase in loss, which after enough epochs leads to further overall decrease. Overall, the model shows improved generalization due to training on augmented data and the addition of dropout in the convolutional layers, indicating that it is learning effectively from the augmented data. Probably, if we trained for more epochs, we could have achieved even better results.

IX. Summary

In this final section we will plot the confusion matrices, per class accuracy and comparison between the actual and the predicted labels for random samples for the best MLP and CNN models obtained from the experiments above.

A. Best MLP Model

The best MLP model achieved an accuracy of 58.65% on the CIFAR-10 test set. Below is the confusion matrix for this model:

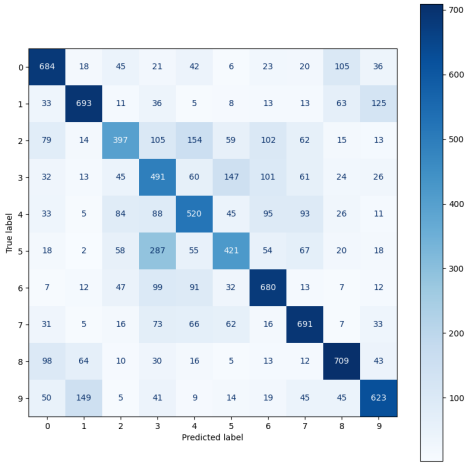


Fig. 22. Confusion Matrix for the best MLP model.

The per class accuracy for the best MLP model is as follows:

- Accuracy of plane: 68.40 % (684/1000)
- Accuracy of car : 69.30 % (693/1000)
- Accuracy of bird : 39.70 % (397/1000)
- Accuracy of cat : 49.10 % (491/1000)
- Accuracy of deer : 52.00 % (520/1000)
- Accuracy of dog : 42.10 % (421/1000)
- Accuracy of frog : 68.00 % (680/1000)
- Accuracy of horse: 69.10 % (691/1000)
- Accuracy of ship : 70.90 % (709/1000)
- Accuracy of truck: 62.30 % (623/1000)

Below are some random samples from the test set along with their actual and predicted labels by the best MLP model:

B. Best CNN Model

The best CNN model achieved an accuracy of 87.4% on the CIFAR-10 test set. Below is the confusion matrix for this model:

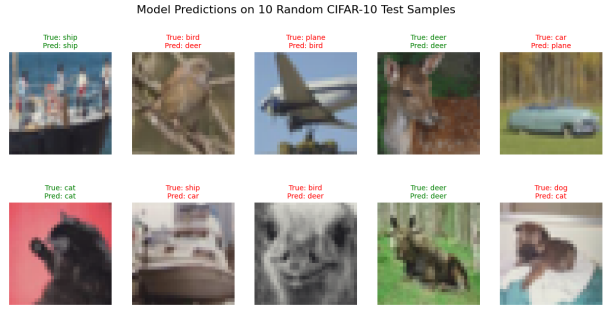


Fig. 23. Random samples from the test set with actual and predicted labels by the best MLP model.

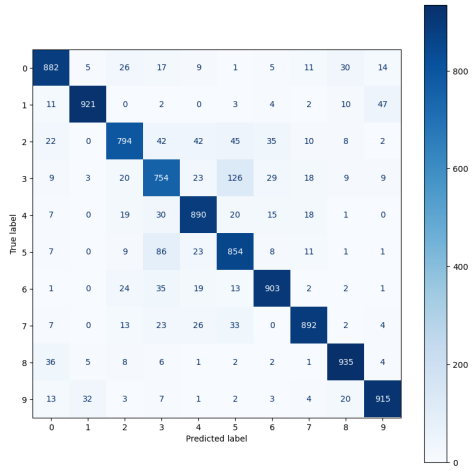


Fig. 24. Confusion Matrix for the best CNN model.

Below are some random samples from the test set along with their actual and predicted labels by the best CNN model:

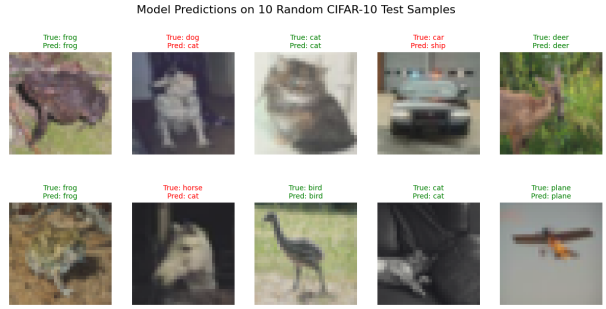


Fig. 25. Random samples from the test set with actual and predicted labels by the best CNN model.

The per class accuracy for the best CNN model is as follows:

- Accuracy of plane: 88.20 % (882/1000)
- Accuracy of car : 92.10 % (921/1000)
- Accuracy of bird : 79.40 % (794/1000)
- Accuracy of cat : 75.40 % (754/1000)
- Accuracy of deer : 89.00 % (890/1000)
- Accuracy of dog : 85.40 % (854/1000)
- Accuracy of frog : 90.30 % (903/1000)
- Accuracy of horse: 89.20 % (892/1000)

- Accuracy of ship : 93.50 % (935/1000)
- Accuracy of truck: 91.50 % (915/1000)

C. Final Comparison

As we can see from the results in this report, the best CNN model significantly outperforms the best MLP model and both best-performing NearestCentroid and K-Nearest Neighbors classifiers on the CIFAR-10 dataset. To summarize:

- NearestCentroid Classifier highest Accuracy: 28.12, total time: 0.45 sec
- 1-Nearest Neighbors Classifier Accuracy: 38.64, total time: 106 sec (PCA included)
- 3-Nearest Neighbors Classifier Accuracy: 37.94, total time : 108 sec (PCA included)
- Best MLP Model Accuracy: 58.65, training time: 1 hour, 18 minutes, 54 sec
- Best CNN Model Accuracy: 87.4, training time: 2 hours, 9 minutes, 49 sec

NearestCentroid ranks last among all models, achieving an accuracy of only 28.12%. but is the fastest to train and predict. (does not require training or PCA compared to k-NN). The K-Nearest Neighbors classifiers perform better, with the 1-NN model achieving an accuracy of 38.64% and the 3-NN model achieving 37.94%. However, both k-NN models are significantly slower during prediction due to the need to compute distances to all training samples and in some cases require PCA to reduce dimensionality. The MLP model shows a substantial improvement over the traditional classifiers, achieving an accuracy of 58.65%, but is significantly slower to train compared to both NearestCentroid and k-NN classifiers. Finally, the CNN model outperforms all other models by a wide margin, achieving an impressive accuracy of 87.4%, but also requires the longest training time.